```
> SYSTEM ROM v3.1.4 ................................ [OK]
> PHOSPHOR DISPLAY SUBSYSTEM ....................... [OK]
> MOUNTING /dev/disk1s1 (APFS, sealed) ............. [OK]
> LOADING artifact index [09 DOMAINS] .............. [OK]
> VERIFYING command corpus ......................... [OK]
> OPERATOR CLEARANCE ............................... [GRANTED]
```

MACOS THREAT HUNTING & FORENSICS

ARTIFACT-INDEXED FIELD GUIDE

>> A C C E S S G R A N T E D <<

BUILD v1.0 STAMP 2026.07.29
SCOPE native macOS CLI · osquery · YARA
ORDER by artifact domain, not by kill-chain phase
READER operator-level. no primers, no framing.

>> INDEX

[01] PROCESS & EXECUTION .....
[02] PERSISTENCE .....
[03] NETWORK .....
[04] FILESYSTEM & METADATA .....
[05] OBFUSCATION & SPOOFING .....
[06] LOGS & UNIFIED LOGGING .....
[07] OSQUERY .....
[08] YARA .....
[09] QUICK TRIAGE CHECKLIST .....


```
> run --section 01 ▉
```

NO WARRANTY · VERIFY EVERY COMMAND AGAINST YOUR OWN BUILD

[01] > PROCESS & EXECUTION

>> ENUMERATION

```
ps -axo pid,ppid,uid,user,lstart,etime,%cpu,%mem,args — full process table with true start time and ancestry
ps -axo pid,ppid,args | awk '$2==1 && $1!=1' — orphaned / launchd-reparented processes
pgrep -fl <pattern> — match on full argv, not just comm
launchctl procinfo <pid> — launchd view: job label, domain, path, sandbox, entitlement flags
launchctl print pid/<pid> — live job state, environment, spawn path
lsappinfo list — registered GUI apps + ASN, bundle path, launch reason
lsappinfo info -only bundlepath <asn> — resolve ASN to on-disk bundle
lsof -p <pid> — open files, sockets, dylibs, cwd for one process
lsof -c <name> -nP — everything opened by processes matching a name
vmmap <pid> | grep -i 'mapped file' — mapped images incl. injected/non-bundled dylibs
sample <pid> 5 -f /tmp/s.txt — 5s stack sample; call paths of a suspicious process
ps -axo pid,args | grep -Ei 'base64|curl|osascript|-e ' — argv-visible staging and living-off-the-land invocation
```

>> CODE SIGNATURE / NOTARIZATION

```
codesign -dv --verbose=4 <path> — identifier, TeamIdentifier, CDHash, flags, Authority chain
codesign -vvv --deep --strict <path> — validate seal across nested bundles
codesign -d --entitlements :- --xml <path> — dump entitlements plist
codesign -d -r- <path> — designated requirement string
codesign -dv <path> 2>&1 | grep -E 'Signature=adhoc|not signed' — ad-hoc or unsigned in one line
spctl -a -vvv -t exec <path> — Gatekeeper assessment for execution
spctl -a -vvv -t install <dmg|pkg> — assessment for installer/disk image
spctl --status ; csrutil status — Gatekeeper and SIP enablement state
stapler validate <path> — notarization ticket stapled to the bundle?
syspolicy_check distribution <path> — full notarization / signing verdict (14+)
```

!! Signature=adhoc + no TeamIdentifier + quarantine bit absent = strong triage lead.

>> SWEEPS

```
find /Applications ~/Applications -maxdepth 3 -name '*.app' -exec codesign -vv {} \; 2>&1 | grep -v 'valid on disk' — every app that fails signature validation
find /usr/local/bin /tmp /var/tmp -type f -perm -u+x -exec codesign -dv {} \; 2>&1 | grep -B1 'not signed' — unsigned executables in writable paths
system_profiler SPInstallHistoryDataType — package install history with dates and source
system_profiler SPApplicationsDataType | grep -A4 'Obtained from: Unknown' — apps with no identifiable signing origin
```

[02] > PERSISTENCE

>> LAUNCHD

ls -la@ ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons — user + system job plists incl. xattrs
launchctl list | awk '$1!="-"' — jobs currently loaded with a live PID
launchctl print gui/$UID | less — full user domain: jobs, endpoints, env, paths
launchctl print system | less — full system domain (root)
launchctl dumpstate > /tmp/lds.txt — complete launchd state snapshot
launchctl print-disabled system — disabled / overridden jobs — security-agent masking
plutil -p <plist> — readable dump of binary or XML plist
plutil -convert xml1 -o - <plist> — normalise binary plist to XML for grep/diff
grep -rlE 'RunAtLoad|KeepAlive' /Library/Launch* ~/Library/LaunchAgents — auto-start and self-respawning jobs
plutil -p /Library/Launch*/*.plist | grep -A3 ProgramArguments — program targets across every system job

!! Compare label vs filename vs ProgramArguments[0] — mismatch is a classic tell.

>> LOGIN ITEMS / BTM

sfltool dumpbtm — Background Task Management db: agents, daemons, login items, dev tools (13+, root)
osascript -e 'tell app "System Events" to get name of every login item' — legacy login items
ls -la ~/Library/Application\ Support/com.apple.backgroundtaskmanagementagent/ — BTM store on disk
defaults read com.apple.loginwindow — LoginHook / LogoutHook / auto-login artefacts

>> CRON, PERIODIC, AT

crontab -l ; sudo crontab -l -u root — per-user cron tables
ls -la /usr/lib/cron/tabs/ /etc/cron* — on-disk crontabs and drop-ins
ls -la /etc/periodic/{daily,weekly,monthly}/ — periodic script directories — rarely audited
atq ; ls -la /var/at/jobs/ — queued at(1) jobs
cat /etc/zshenv /etc/zprofile ~/.zshrc ~/.zprofile ~/.bash_profile 2>/dev/null — shell rc persistence (/etc/zshenv fires for non-interactive too)

>> TCC / PPPC / PROFILES

sudo sqlite3 /Library/Application\ Support/com.apple.TCC/TCC.db 'select service,client,auth_value from access;' — system TCC grants (FDA)
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db 'select service,client,auth_value from access;' — user grants — ScreenCapture / AppleEvents / SystemPolicyAllFiles
tccutil reset All <bundle-id> — revoke all grants for a bundle
profiles show -type configuration — installed configuration profiles (user)
sudo profiles list -all -verbose — all profiles incl. device-scoped, with payload UUIDs
ls -la /Library/Managed\ Preferences/ /var/db/ConfigurationProfiles/Store/ — MDM-delivered preference enforcement

!! A profile granting PPPC/AppleEvents to a non-MDM bundle id is high-signal.

>> OTHER LOAD POINTS

kmutil showloaded --list-only — loaded kexts (kextstat is deprecated)
systemextensionsctl list — installed system extensions / ES clients
ls -la /Library/SecurityAgentPlugins/ /Library/ScriptingAdditions/ — authorization plugins and OSAX injection points
ls -la /Library/QuickLook ~/Library/QuickLook /Library/Spotlight — QL/MD plugins execute on preview and index
ls -la /etc/emond.d/rules/ 2>/dev/null — legacy emond rules (removed on 13+, present on older estates)
dscl . -list /Users UserShell ; dscl . -read /Users/<u> AuthenticationAuthority — rogue accounts, shell and auth changes
ls -la ~/.ssh/authorized_keys /var/root/.ssh/ — key-based backdoor access

[03] > NETWORK

>> CONNECTIONS & LISTENERS

`lsof -i -P -n` — all IP sockets with owning process
`lsof -i -P -n | grep LISTEN` — listening sockets only
`lsof -nP -i -a -p <pid>` — sockets for a single process
`lsof -nP -i @<ip>` — every process talking to one address
`netstat -anv -p tcp` — TCP table with queue depths and PIDs
`netstat -anv -p udp` — UDP endpoints
`nettop -P -l 1 -x` — per-process byte counters, one sample
`nettop -m tcp -P -l 1` — per-process TCP with remote endpoints
`sudo tcpdump -i pktap,all -k NP -n -w /tmp/cap.pcap` — capture with per-packet process attribution (pktap)

>> DNS

`scutil --dns` — resolver configuration and search order per interface
`sudo killall -INFO mDNSResponder` — dump resolver cache + state into the unified log
`log show --predicate 'process == "mDNSResponder"' --last 30m --info` — read the dumped cache and query stream
`dns-sd -B _services._dns-sd._udp` — enumerate advertised mDNS/Bonjour services
`dscacheutil -statistics` — cache hit/miss counters (cache dump removed in modern macOS)
`cat /etc/hosts ; cat /etc/resolver/* 2>/dev/null` — static overrides and split-DNS redirection

>> FILTER / ROUTING / PROXY

`sudo pfctl -s info` — pf enabled state and counters
`sudo pfctl -s rules -vv` — active pf ruleset with per-rule counters
`sudo pfctl -s states` — current pf state table
`sudo pfctl -s nat ; sudo pfctl -a '*' -sr` — NAT/redirect rules and all anchors (incl. injected anchors)
`cat /etc/pf.conf ; ls -la /etc/pf.anchors/` — on-disk ruleset and anchor files
`netstat -rn ; route -n get default` — routing table and default gateway
`arp -an` — ARP cache — spoofing / rogue gateway
`scutil --proxy` — system proxy settings incl. PAC URL
`networksetup -listallnetworkservices ; networksetup -getwebproxy "Wi-Fi"` — per-service proxy configuration
`sudo /usr/libexec/ApplicationFirewall/socketfilterfw --listapps` — ALF per-app inbound rules
`networksetup -listallhardwareports ; ifconfig -a` — interfaces, incl. unexpected utun/tap/bridge

!! utunN with no VPN profile, or a new bridge/tap device, warrants a look.

[04] > FILESYSTEM & METADATA

>> EXTENDED ATTRIBUTES

xattr -l <path> — all xattrs with values
ls -l@O <path> — xattr names + BSD flags (uchg/schg/hidden) inline
xattr -p com.apple.quarantine <path> — flags;timestamp(hex);agent;event-UUID
xattr -p com.apple.provenance <path> — provenance record tying binary to installing app (13+)
xattr -p com.apple.macl <path> — TCC file-access grant bound to a bundle
xattr -p com.apple.metadata:kMDItemWhereFroms <path> | xxd -r -p | plutil -p - — source URL chain of a download
xattr -rl ~/Downloads /tmp /Users/Shared — recursive xattr sweep of drop directories
find ~ -type f -exec xattr -p com.apple.quarantine {} \; -print 2>/dev/null — every quarantined file under a home dir
sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 'select datetime(LSQuarantineTimeStamp+978307200,"unixepoch"),LSQuarantineAgentName,LSQuarantineDataURLString from LSQuarantineEvent order by 1 desc limit 50;' — download provenance ledger — URL, agent, timestamp

!! Absent quarantine xattr on a file in ~/Downloads means it was written by something that isn't a quarantine-aware app, or was stripped.

>> SPOTLIGHT METADATA

mdls <path> — all MD attributes incl. kMDItemDateAdded, WhereFroms, ContentType
mdfind "kMDItemWhereFroms == '*'" -onlyin ~ — everything Spotlight records as downloaded
mdfind "kMDItemContentType == 'public.unix-executable'" -onlyin ~ — executables in user-writable space
mdfind -onlyin / "kMDItemFSCreationDate >= \$time.today(-1)" — files created in the last day
mdutil -s / ; mdutil -sa — index status per volume — indexing disabled = anti-forensics

>> TIMESTAMPS (MACB)

stat -f 'M:%Sm A:%Sa C:%Sc B:%SB %N' -t '%F %T' <path> — mtime / atime / ctime / birthtime in one line
find <dir> -type f -newerct '2026-07-01' -print0 | xargs -0 stat -f '%Sc %N' — inode-change-time window (survives touch of mtime)
find / -type f -mtime -1 -not -path '/System/*' -not -path '/dev/*' 2>/dev/null — recently modified outside the sealed system volume

!! ctime < mtime, or birthtime > mtime, means timestomping. Compare against kMDItemDateAdded.

>> HIDDEN CONTENT / VOLUMES

ls -laO ; ls -a@ — dotfiles, BSD flags, hidden flag, xattr markers
find / -flags +hidden -not -path '/System/*' 2>/dev/null — files hidden via chflags rather than a leading dot
ls -l <file>/..namedfork/rsrc — resource fork content (non-zero size = payload carrier)
find . -name '._*' -o -name '.DS_Store' — AppleDouble carriers and Finder metadata
hdiutil info — currently attached disk images with backing store paths
mount ; diskutil list ; diskutil apfs list — mounted volumes, APFS containers, hidden volumes
tmutil listlocalsnapshots / — APFS snapshots — pre-incident state to diff against
log show --predicate 'process == "diskarbitrationd"' --last 7d — historical mount/attach events for images and externals

[05] > OBFUSCATION & SPOOFING

>> MACH-O / FAT BINARY INSPECTION

file <bin> — arch list, fat/universal, Mach-O type
lipo -detailed_info <bin> — per-slice arch, offset, size, align — hidden or mismatched slices
lipo -thin arm64 <bin> -output /tmp/slice — extract one slice for isolated analysis
otool -hv <bin> — Mach-O header flags (PIE, restrict, no-heap-exec)
otool -L <bin> — linked dylibs — @rpath/@executable_path hijack surface
otool -l <bin> | grep -A4 LC_RPATH — rpath search order for dylib planting
otool -l <bin> | grep -A5 LC_ENCRYPTION_INFO — encrypted text segment (cryptid 1 = packed)
otool -l <bin> | grep -A3 __RESTRICT — __RESTRICT segment blocking DYLD_INSERT_LIBRARIES
nm -mu <bin> — undefined symbols — imported API surface
strings -a -n 8 <bin> | grep -Ei 'http|/tmp|osascript|curl|base64' — embedded C2, staging paths, interpreter calls
codesign -d --entitlements :- --xml <bin> | plutil -p - — entitlements: get-task-allow, disable-library-validation, TCC-adjacent keys

!! disable-library-validation + get-task-allow on a shipping app = injection-ready.

>> NAME & IDENTITY SPOOFING

ls | LC_ALL=C grep '[^ -~]' — non-ASCII filenames — homoglyph and RTL-override candidates (BSD grep: no -P)
ls | perl -CS -ne 'print if /[\x{202A}-\x{202E}\x{2066}-\x{2069}]/' — U+202A-202E / U+2066-2069 bidi + RLO extension spoofing
ls | perl -CS -ne 'print if s/([^\x00-\x7F])/sprintf"<U+%04X>",ord$1/ge' — rewrite suspicious names with codepoints made explicit
printf '%s' "<name>" | hexdump -C — raw bytes of a filename
plutil -p <app>/Contents/Info.plist | grep -E 'CFBundleIdentifier|CFBundleExecutable|LSUIElement' — bundle id vs. signing identity; LSUIElement hides from Dock
codesign -dv <app> 2>&1 | grep Identifier ; plutil -extract CFBundleIdentifier raw <app>/Contents/Info.plist — signing identifier vs declared bundle id — mismatch is deliberate

!! Also check: app name matching a system binary but living outside /System or /usr/bin.

>> LOLBINS — NATIVE EXECUTION & TRANSFER

osascript -e ... / osascript -l JavaScript — AppleScript + JXA: scripting-bridge execution, TCC pivot
osacompile -o x.app -e '...' — compile a script into an app bundle on the fly
curl -o / nscurl / xattr -d com.apple.quarantine — fetch payload, then strip the quarantine bit
sqlite3 <db> '.shell <cmd>' — shell escape from within sqlite
openssl enc -d -aes-256-cbc -in x -out y ; openssl base64 -d — native decrypt/decode of staged payloads
plutil / defaults write — write persistence without touching a text editor
security find-generic-password -w -s <svc> ; security dump-keychain — credential access via native keychain CLI
sysadminctl / dscl . -create — account creation and privilege change
screencapture -x ; sysdiagnose — silent capture and bulk host data collection
tclsh / php / perl / awk / mktemp / streamzip — surviving interpreters — python/ruby removed from base OS at 12.3
xattr -c -r <path> — mass quarantine stripping — audit for this specific invocation

>> PLIST & PAYLOAD OBFUSCATION

plutil -convert xml1 -o - <plist> | grep -Ei 'base64|bash -c|eval|\\x' — encoded or chained commands inside job definitions
file <plist> — binary plist masquerading as another type
plutil -lint <plist> — malformed plists that still load
grep -aoE '[A-Za-z0-9+/]{60,}={0,2}' <file> | base64 -d | file - — extract and identify long base64 blobs
grep -aoE '(\\x[0-9a-f]{2}){8,}' <file> — hex-escaped payload strings
log show --last 24h --info --predicate 'eventMessage CONTAINS[c] "base64" OR eventMessage MATCHES ".*[A-Za-z0-9+/]{80,}.*"' — encoded payloads and long opaque blobs in the log stream

[06] > LOGS & UNIFIED LOGGING

>> MECHANICS

log show --last 1h --info --debug — include info/debug levels — default omits both
log show --style ndjson --last 1h > /tmp/l.ndjson — machine-parsable output for downstream tooling
log show --style syslog --last 1h — compact one-line-per-event form
log collect --output /tmp/h.logarchive --last 3d — package the log store for offline analysis
log show --archive /tmp/h.logarchive --predicate '...' — query a collected archive
log stream --predicate '...' --level debug — live tail
log config --status — current logging mode / whether private data is redacted

!! <private> redaction hides the interesting fields. Note it, don't fight it on a live host.

>> PREDICATES BY CATEGORY

--predicate 'subsystem == "com.apple.xpc.launchd"' — job load/unload/spawn — persistence installation (§02)
--predicate 'process == "launchd" AND eventMessage CONTAINS "Service"' — service registration events
--predicate 'subsystem == "com.apple.syspolicy"' — Gatekeeper assessments, notarization checks (§01)
--predicate 'process == "syspolicyd" AND eventMessage CONTAINS[c] "deny"' — blocked executions
--predicate 'subsystem == "com.apple.TCC"' — TCC prompts, grants, and denials (§02)
--predicate 'process == "tccd" AND eventMessage CONTAINS "AUTHREQ"' — authorisation requests with requesting client
--predicate 'process CONTAINS "XProtect"' — XProtect / XProtect Remediator scan and detection results
--predicate 'subsystem == "com.apple.securityd"' — keychain and trust-evaluation activity
--predicate 'process == "mDNSResponder"' — DNS queries and resolver state (§03)
--predicate 'subsystem == "com.apple.network"' — connection establishment via Network.framework
--predicate 'process == "kernel" AND eventMessage CONTAINS "Sandbox"' — sandbox violations
--predicate 'eventMessage CONTAINS "com.apple.quarantine"' — quarantine set/clear events (§04)
--predicate 'process == "diskarbitrationd"' — disk image attach / volume mount history (§04)
--predicate 'eventMessage CONTAINS[c] "osascript" OR eventMessage CONTAINS[c] "curl"' — LOLBin invocation traces (§05)

>> LIVE SYSCALL / FILE TRACING

sudo fs_usage -w -f filesys — all filesystem syscalls, wide format
sudo fs_usage -w -f exec — exec/posix_spawn events — the native execsnoop
sudo fs_usage -w -f pathname <proc> — path-level activity scoped to one process
sudo fs_usage -w -f network — socket-layer syscalls
sudo fs_usage -w -e Terminal fs_usage — exclude your own tooling from the trace
sudo opensnoop -n <proc> / sudo execsnoop — DTrace equivalents — require SIP disabled
sudo eslogger exec fork mount | jq . — EndpointSecurity event stream, JSON (13+, FDA required)
sudo eslogger create rename unlink — file lifecycle events via ES

!! eslogger is the closest thing to native EDR telemetry on modern macOS. Prefer it over DTrace.

>> LEGACY & AUXILIARY LOGS

cat /var/log/install.log — package and OS install activity, survives reboots
last -50 ; who -a ; w — login history from utmpx
sudo praudit -l /var/audit/current — BSM audit trail (deprecated; still present on older estates)
ls -la ~/.zsh_history ~/.bash_history /var/root/.*history — shell history, incl. root's
log show --predicate 'eventMessage CONTAINS "sudo"' --last 7d --info — sudo invocations across the log store

[07] > OSQUERY

>> §01 PROCESS & EXECUTION

```
processes — SELECT pid,name,path,cmdline,parent,uid,start_time FROM processes WHERE on_disk=0;
signature — SELECT path,signed,identifier,authority FROM signature WHERE path='/path/bin';
process_envs — SELECT pid,key,value FROM process_envs WHERE key LIKE 'DYLD%';
process_open_files — SELECT p.name,f.path FROM process_open_files f JOIN processes p USING(pid);
hash — SELECT path,sha256 FROM hash WHERE path='/path/bin';
apps — SELECT name,bundle_identifier,path,last_opened_time FROM apps ORDER BY last_opened_time DESC;
```

>> §02 PERSISTENCE

```
launchd — SELECT label,path,program,program_arguments,run_at_load,keep_alive FROM launchd;
launchd_overrides — SELECT label,key,value FROM launchd_overrides WHERE key='Disabled';
startup_items — SELECT name,path,type,source,status FROM startup_items;
crontab — SELECT command,path,minute,hour FROM crontab;
system_extensions — SELECT path,identifier,team,category,state FROM system_extensions;
kernel_extensions — SELECT name,version,linked_against FROM kernel_extensions;
managed_policies — SELECT domain,name,value,username FROM managed_policies;
authorized_keys — SELECT * FROM users JOIN authorized_keys USING(uid);
shell_history — SELECT u.username,h.command FROM users u JOIN shell_history h USING(uid);
sudoers — SELECT source,header,rule_details FROM sudoers;
```

>> §03 NETWORK

```
listening_ports — SELECT l.port,l.protocol,l.address,p.name,p.path FROM listening_ports l JOIN
  processes p USING(pid);
process_open_sockets — SELECT p.name,s.remote_address,s.remote_port FROM process_open_sockets s JOIN
  processes p USING(pid) WHERE s.remote_port!=0;
alf / alf_exceptions — SELECT * FROM alf_exceptions;
dns_resolvers — SELECT id,type,address FROM dns_resolvers;
etc_hosts — SELECT address,hostnames FROM etc_hosts;
interface_addresses — SELECT interface,address,mask FROM interface_addresses;
```

>> §04 FILESYSTEM & METADATA

```
extended_attributes — SELECT path,key,value FROM extended_attributes WHERE key='com.apple.quarantine'
  AND directory='/Users/x/Downloads';
file — SELECT path,btime,ctime,mtime,mode,uid FROM file WHERE directory='/tmp';
mdfind — SELECT path FROM mdfind WHERE query="kMDItemWhereFroms == '*'";
disk_events — SELECT action,path,name,vendor,checksum,time FROM disk_events;
time_machine_destinations — SELECT alias,destination_id,encryption FROM time_machine_destinations;
```

>> §05 OBFUSCATION & SPOOFING / POSTURE

```
plist — SELECT key,value FROM plist WHERE path='/Library/LaunchDaemons/x.plist';
preferences — SELECT domain,key,value FROM preferences WHERE domain='com.apple.loginwindow';
gatekeeper — SELECT assessments_enabled,dev_id_enabled,version FROM gatekeeper;
sip_config — SELECT config_flag,enabled FROM sip_config;
xprotect_entries / xprotect_reports — SELECT name,description,user_action,time FROM xprotect_reports;
certificates — SELECT common_name,issuer,not_valid_after,path FROM certificates WHERE path LIKE
  '%System.keychain';
safari_extensions / chrome_extensions — SELECT name,identifier,path,author FROM chrome_extensions;
es_process_events — SELECT pid,path,cmdline,signing_id,parent FROM es_process_events; -- requires ES
  publisher enabled
unified_log — SELECT message,subsystem,category FROM unified_log WHERE subsystem='com.apple.TCC';
```

!! osqueryi --line for readable output; .schema <table> for column truth on your build.

[08] > YARA

>> INVOCATION

yara -r -s -w rules.yar <path> — recursive scan, show matched strings, suppress warnings
yara -r -f -N rules.yar <path> — fast-match mode, skip symlinks — bulk sweeps
yara -m -e rules.yar <path> — print rule metadata and namespace with each hit
yara -t <tag> rules.yar <path> — run only rules carrying a tag
yara -d var=value rules.yar <path> — supply external variables (e.g. filepath, filename)
yara -p 8 -r rules.yar <path> — 8 scanner threads
yara --max-strings-per-rule=0 --stack-size=32768 rules.yar <path> — raise limits for large/complex rulesets
yarac rules.yar rules.compiled ; yara -C rules.compiled <path> — precompile once, reuse across hosts
yara rules.yar <pid> — scan live process memory by PID (root)
for p in $(ps -axo pid=); do yara mem.yar $p 2>/dev/null; done — sweep all process memory

>> SCAN TARGETS — RANKED

~/Downloads ~/Desktop ~/Library/Containers/*/Data/Downloads — initial-access landing zones (§04 quarantine ledger)
/tmp /var/tmp /private/var/folders/*/*/T /Users/Shared — staging and drop directories
~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons — persistence plists and their ProgramArguments targets
~/Library/Application\ Support /Library/Application\ Support — payload homes for launchd jobs
/usr/local/bin /opt ~/bin — user-writable executable paths outside SIP
~/Library/Preferences /Library/Preferences — plist-based config and encoded payload storage
/Volumes/* — freshly attached DMGs before contents are copied

!! The sealed system volume is read-only and SIP-protected — deprioritise /System and /usr/bin.

>> MODULES WORTH USING ON MACOS

import "macho" — macho.number_of_segments, macho.entry_point, macho.dylibs, fat slice iteration
macho.dylibs[i].name / macho.rpaths[i].path — linked libraries and rpath search order as rule conditions
macho.entry_point, macho.filetype, macho.flags — header-level triage (MH_BUNDLE, MH_DYLIB, PIE flags)
import "hash" — hash.sha256(0, filesize) — hash-anchored rules and section-scoped hashing
import "math" — math.entropy(0, filesize) > 7.2 — packed/encrypted content detection
filesize, uint32(0) == 0xfeedfacf — condition anchors: size bounds and Mach-O 64 magic

!! 0xcafebabe = fat header (also Java class) — gate on filesize or slice count to cut FPs.

[09] > QUICK TRIAGE CHECKLIST

>> FIRST PASS — ONE COMMAND PER DOMAIN

[1] `ps -axo pid,ppid,user,lstart,args | grep -vE '^\s*[0-9]+\s+1 ' | less` — process tree with true start times

[1] `find /Applications ~/Applications -maxdepth 3 -name '*.app' -exec codesign -vv {} \; 2>&1 | grep -v 'valid on disk'` — every app failing signature validation

[2] `sudo sfltool dumpbtm` — complete background-item inventory (13+)

[2] `ls -la@ ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons && launchctl list | awk '$1!="-"'` — persistence on disk vs. loaded in launchd

[3] `lsof -i -P -n | grep -E 'LISTEN|ESTABLISHED'` — live listeners and outbound sessions with owning process

[3] `sudo pfctl -s rules -vv ; scutil --proxy ; cat /etc/hosts` — filter, proxy and resolver tampering

[4] `sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 'select datetime(LSQuarantineTimeStamp+978307200,"unixepoch"),LSQuarantineAgentName,LSQuarantineDataURLString from LSQuarantineEvent order by 1 desc limit 25;'` — last 25 downloads with source URL and agent

[4] `find / -type f -mtime -2 -not -path '/System/*' -not -path '/dev/*' 2>/dev/null | head -200` — recent writes outside the sealed volume

[5] `otool -L <suspect> ; codesign -d --entitlements :- --xml <suspect> | plutil -p -` — link surface and entitlement posture of the lead binary

[5] `ls -R ~/Downloads /tmp | LC_ALL=C grep '[^ -~]'` — non-ASCII filenames in drop directories

[6] `log show --last 24h --info --predicate 'subsystem == "com.apple.syspolicy" OR subsystem == "com.apple.TCC" OR subsystem == "com.apple.xpc.launchd"'` — Gatekeeper + TCC + launchd in a single pass

[6] `sudo eslogger exec | jq -r '[.time,.process.executable.path,.event.exec.args[0]]|@tsv'` — live execution telemetry while you work

[7] `osqueryi --line "SELECT label,program,program_arguments,run_at_load FROM launchd WHERE run_at_load=1;"` — auto-start jobs, structured

[8] `yara -r -f -w -C rules.compiled ~/Downloads /tmp /var/tmp ~/Library/LaunchAgents /Library/Launch*` — sweep the ranked targets in one invocation

>> POSTURE — CONFIRM BEFORE YOU TRUST ANY OF THE ABOVE

`csrutil status ; spctl --status ; log config --status` — SIP, Gatekeeper, and log redaction state

`sudo profiles list -all | grep -i payload` — configuration profiles that may be overriding policy

`mdutil -sa` — Spotlight indexing state — disabled indexes break §04 mdfind queries

!! If SIP is off or profiles are enforcing policy, treat every on-host result as advisory and pull the disk.